(1)

OBJECT-ORIENTED GOOD PRACTICES

Conceptual Modeling:

- What: capturing ideas and concepts in a form that supports reasoning, analysis, prontication and design.
- Why: to understand a real-world system, visualize it as it is or as we want it to become, document key decisions, specify a solution, guide construction, and above all communicate efficiently.
- When: at any stage of development, starting at the very beginning: analysis, design, specification, implementation and deployment.

UML (Unified Modeling Language)
UML is a collection of modeling languages. Its main diagrams are:
- Use case diagrams: who needs the system and what their goals are.
- Class diagrams: the system's classes and how they relate to each other.
- Interaction diagrams: how objects communicate with each other.
- State charts: the internal behavior of an object or a method.

Good Practices:
1. Encapsulation: Keep attributes private, because public class variables break this principle.
2. Loosely coupled layers: organize the system in layers: user interface → application logic → persistence / communications → operating system. Each layer gives a clear abstraction to the layer above it and only uses the layers below it.
3. Prefer aggregation over inheritance:
   - Aggregation means A contains B. Inheritance means A is a B.
   - Aggregation is more flexible because it can change at runtime.
   - Composition is a stronger form of aggregation.
4. Program to an interface or abstraction: this reduces coupling and removes unnecessary dependencies. A class's interface is made up of its public methods and properties.

(2)

INTRODUCTION TO UML

What UML is:
UML is a semi-formal, expressive object-oriented modeling language. It works with any development process, supports incremental modeling (elements can be hidden or left incomplete), and can be extended. It was standardized in the 90s to unify competing methods.
Building blocks:
- Elements are like word: classes, objects, messages, states, packages, comments, and so on.
- Diagrams are like sentences.
- Views are like chapters: use-case, logical, implementation, process, deployment.

VERSATIL

Scanned with CamScanner

• Models are like books.

Three perspectives
- Analysis: understand the real-world problem as it is.
- Specification: define what the system will become.
- Implementation: describe the actual software classes.

Diagram types:
- Structural (static): class, object (a snapshot of instances), component, deployment (hardware nodes).
- Behavioral (dynamic): use case (actors and goals), sequence (messages over time), collaboration (links between objects), statechart (states and events), activity (flow of control).

Key tips:
- Diagrams exist to communicate, not to look pretty.
- Avoid redudant diagrams but don't oversimplify.
- Balance structural and behavioral diagrams.
- Use clear names and keep diagrams organized.

(3)

USE CASES

Actors
Actors are anything outside the system that interacts with it: users, other software systems, or hardware such as sensors. Each actor has goals with respect to the system. Actors are named in singular and drawn as stick figures. All users are actors, but not all actors are users.

Use cases
A use case is a goal the user needs to achieve, written independently of any software feature. It's name should state the goal without design choices. Each concrete scenario that achieves the goal is an instance of the use case.
- Example: "Allow only authorized fcwlty and staff to update student records".

Use cases vs system interactions
- Use cases answer what and why.
- System interactions answer how, as step-by-step actions such as select course → select student → enter grade → confirm.
- Mixing them up hides why features are needed and limits creativity.

Use case diagrams.
They have four main components: actors, use cases, relationships, and subjets (the system boundary). They can also include notes, constraints and packages.

Relationships.
- Actor generalization: one actor is a subset of another, for example Graduate Student → Student.

Scanned with CamScanner

- <<extend>>: a variation of another use case that applies under a condition at an extension point, for example altering the grade of a class taken more than a year ago.
- <<include>>: one use case requires another's goal. It is a form of aggregation and usually has no actor of its own.
- Generalization: one use case is a more specific version of another, for example altering a grade for a graduate course.

Tips.
- Finding actors: ask who use the system, who gives it input, who uses its output, and which external systems or devices it talks to.
- Finding use cases: list the actors and their activities, or list external events and how the system responds. Be patient and keep the diagram high-level.
- A single person, can play several roles, an actor can have many use cases, and a use case can have many actors.

Benefits:
Use cases capture user-visible functions, show who needs the system, reveal opportunities for reuse, and give a basis for planning incremental development and for system testing.

4

OBJECTS, CLASSES AND BASIC CLASS DIAGRAMS

Objects
Objects are the basic building blocks of OO systems. An object represents a "thing", has boundaries in space or time, is an abstraction, encapsulates state and behavior and has its own identity. Events, relationships, behavior and constraints are usually not modeled as objects.

Classes
A class groups objects that share common properties, such as Student, Course or Instructor. Its meaning depends on the perspective:
- Analysis: a set of objects.
- Specification: an interface to software objects.
- Implementation (design): an abstract data type, i.e. class Student {...}.

Class syntax
A class is a box with compartments for name, attributes, operations, responsabilities, and optional user-defined sections.
- Class names: singular nouns in UpperCamelCase, meaningful in the problem domain. Student is better than StudentData. They can include a package path, such as ec.edu.espe.studentsrecord.model.Student.
- Object names: written je: Student, underlined, with real attribute values. If the instance name is left blank, it means any object of that class.

Attributes and operations
- Attribute syntax: [visibility] name [multiplicity] : type = initialValue {property}. Visibility is + public, # protected, - private. Name use lowerCamelCase.

VERZATIL

Scanned with CamScanner

- Operation syntax: [visibility] name (parameters): returnType {property}. Names are verbs in lowerCamelCase.
- In implementation, attributes are data members and operations are methods.

Relationships

- Generalization (empty triangle): "is a." The specialization is a subset of the general class, for example Graduate Student → Student → Person. In code it means inheritance.
- Association (solid line): a structural link between objects, named with a lowercase verb and a reading arrow, such as teaches>.
  - Navigation can be one-way, two-way or unspecified. It matters most in specification and implementation.
  - N-ary association connect three or more classes through a diamond.
- Aggregation (empty diamond): "is part of," for example Slice → Pizza-Order. It is asymmetric and transitive.
- Composition (filled diamond): a stronger aggregation where the part cannot exist without the whole.
- Dependency (dashed arrow): one element uses another, so a change in one may force a change in the other. Common types are «becomes» and «instance of».

Multiplicity

Multiplicity sets how many objects can be linked. It can be a single number (1), many (*), a range (0..8, 1..*), or a combination.

Class diagrams

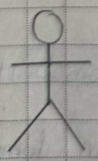
They show the types of objects, their attributes and operations, and their relationships, similar to an ER model but with classes. Each diagram should focus on one topic at a consistent level of abstraction.

- Pitfalls: focusing only on structure and ignoring behavior, mixing perspectives, and using the wrong or mixed levels of abstraction.

(5)

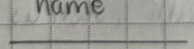
UML QUICK REFENCE

USE CASE DIAGRAMS



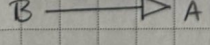
ACTOR

Someone or something outside the system with a goal toward it. Singular noun, each word capitalized: User, Student, Customer.

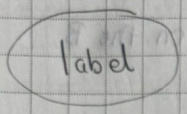


GENERALIZATION

"B is an A": every B is also an A.
(whit) With many specializations, merge the lines into one triangle.

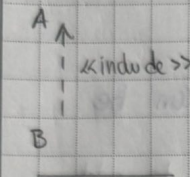


Scanned with CamScanner

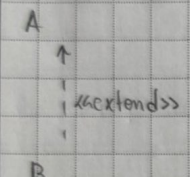


USE CASE
A goal an actor has with respect to the system.
Label it clearly and concisely; it should make sense on its own

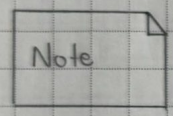
ACTOR HAS USE CASE
Actor A has the goal represented by used case B.
Solid line with no arrowheads or adornments.



INCLUDE
Goal A requires sub-goal B to be met
Break goals into sub-goals, but stop before they describe how the system works.

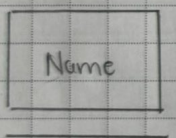


EXTEND
B adds to or changes goal A in special cases.
Use for exceptional situations. The arrow goes from B to A.



NOTE
Any comment, optionally linked to an element by a dotted line.
Jot ideas down, then move them into formal elements later.

CLASS DIAGRAMS



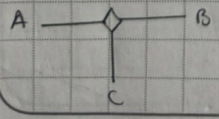
CLASS
A set of real-world objects from the domain.
Singular noun, each word capitalized: Bank Account.

GENERALIZATION
"B is an A": every B object is also an A object.
With many subclasses, merge the lines into one triangle.

NAMED ASSOCIATION
Persistent links meaning "A or B"
Use a precise verb phrase. Avoid just "has".

AGGREGATION
"B is a subpart of A" or "A contains B"
Use only for real part-whole relations, otherwise prefer a named association.

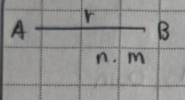
COMPOSITION
Stronger aggregation: if A is destroyed, its B parts are too.
Use when a part belongs to exactly one whole.



N-ARY ASSOCIATION
Links among three or more classes with one meaning
Name it with a full sentence that uses all the class names.

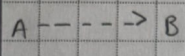
VERZATIL

Scanned with CamScanner



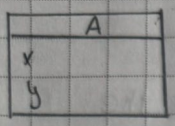
MULTIPLICITY

One A relates to between n and m B objects; written on the B side.
Check all of them before finishing. Use only on associations.



DEPENDENCY

A may depend on B; not a persistent link
Use sparingly in analysis, e.g. to show A as an instance of a template B.



ATTRIBUTES

Properties x and y that every A object has; a value can be another object.
Don't define them too early; turn them into associations to add detail.